

Aim: Write a python program to implement BFS.

Objectives:

The objective of the Breadth-First Search (BFS) algorithm is to traverse or search all the nodes of a graph or tree in a level-by-level manner. It starts from a source node, visits all its neighboring nodes first, and then moves to the next level. BFS is widely used to find the shortest path in an unweighted graph, check whether a path exists between two nodes, and ensure that every reachable node is visited exactly once.

Algorithm:

1. Start.
2. Create an empty queue and a visited array.
3. Insert the starting node into the queue and mark it as visited.
4. While the queue is not empty:
Remove the front node from the queue.
Visit (process) the node.
For each unvisited adjacent node:
Mark it as visited.
Insert it into the queue.
5. Repeat until the queue becomes empty.
6. Stop.

Flow chart:

Source Code:

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}
```

```
visited = set()  
queue = deque()
```

```
start = 'A'  
visited.add(start)  
queue.append(start)
```

```
while queue:  
    node = queue.popleft()  
    print(node, end=" ")
```

```
for neighbor in graph[node]:
    if neighbor not in visited:
        visited.add(neighbor)
        queue.append(neighbor)
```

Result:

The Breadth-First Search (BFS) algorithm was successfully executed. The graph was traversed in level-by-level order, and all reachable nodes were visited exactly once.

Output:



The screenshot shows a Python IDE with a file named 'main.py' and an output window. The code implements a Breadth-First Search (BFS) algorithm on a graph with 6 nodes (A-F). The graph structure is: A is connected to B and C; B is connected to A and E; C is connected to A and D; D is connected to C and E; E is connected to B and D; F is isolated. The algorithm starts at node 'A', visits its neighbors B and C, then their neighbors E and D, and finally F. The output window displays the sequence of visited nodes: A B C D E F, followed by a confirmation message: '*** Code Execution Successful ***'.

```
main.py 1 from collections import deque
2
3 graph = {
4     'A': ['B', 'C'],
5     'B': ['A', 'E'],
6     'C': ['A', 'D'],
7     'D': [],
8     'E': [],
9     'F': []
10 }
11
12 visited = set()
13 queue = deque()
14
15 start = 'A'
16 visited.add(start)
17 queue.append(start)
18
19 while queue:
20     node = queue.popleft()
21     print(node, end=" ")
22
23     for neighbor in graph[node]:
24         if neighbor not in visited:
25             visited.add(neighbor)
26             queue.append(neighbor)
```

Output: A B C D E F
*** Code Execution Successful ***